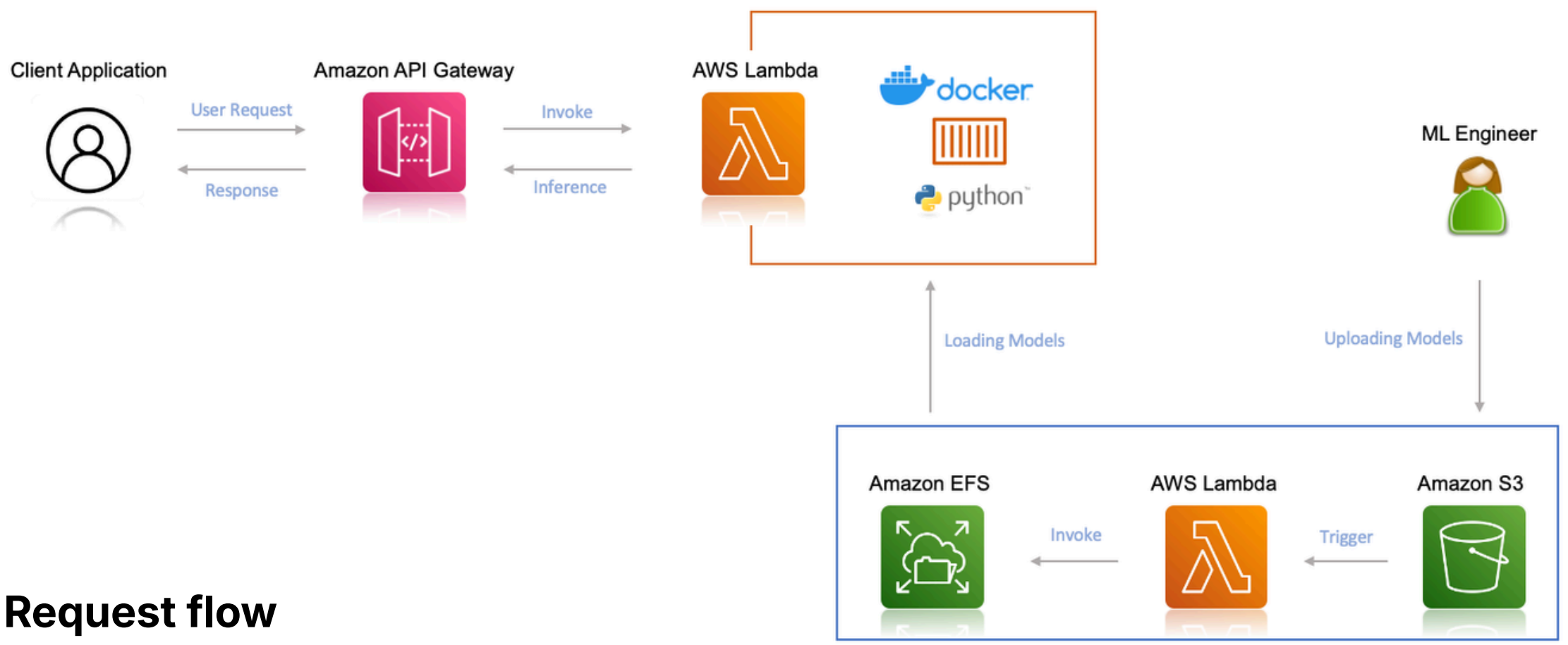
Analytics Vidhya

Guide to Deploy ML Model on AWS

Deploying a machine learning model is often harder than building it. A trained model sitting in a notebook is not useful until it can accept requests and return predictions in real time.

This guide walks through a practical serverless architecture to deploy ML models using Docker, AWS Lambda, and API Gateway. The result is a scalable REST API that serves predictions without managing servers.

Architecture Overview



Request flow

Client → API Gateway → Lambda (Docker Container) → ML Model → **Response**

Components involved:

- Docker – packages the model and dependencies
- Amazon ECR – stores the Docker image
- AWS Lambda – runs the containerized inference code
- API Gateway – exposes the model as a REST API

Step 1: Train and Save the Model

First train your model and save it to disk.

Example using scikit-learn:

```
from sklearn.datasets import load_iris
from sklearn.ensemble import
RandomForestClassifier
import joblib

X, y = load_iris(return_X_y=True)

model = RandomForestClassifier()
model.fit(X, y)

joblib.dump(model, "model.joblib")
```

Now the trained model is stored as:

model.joblib

This file will be packaged inside the Docker container.

Step 2: Create the Inference API

Use FastAPI or Flask to expose a prediction endpoint.

Example using FastAPI.

```python
from fastapi import FastAPI
import joblib
import numpy as np

app = FastAPI()

model = joblib.load("model.joblib")

@app.get("/")
def health():
    return {"status": "running"}

@app.post("/predict")
def predict(features: list):

    data = np.array(features).reshape(1, -1)
    prediction = model.predict(data)

    return {"prediction": int(prediction[0])}
```

This creates two endpoints:

- / → health check
- /predict → returns model predictions

Step 3: Define Dependencies

Create a requirements.txt.

```
fastapi
uvicorn
scikit-learn
joblib
numpy
```

Step 4: Containerize the Application with Docker

Create a Dockerfile.

```
FROM public.ecr.aws/lambda/python:3.10

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY app.py .
COPY model.joblib .

CMD ["app.predict"]
```

This Docker image contains:

- Python runtime
- Model file
- API code
- All dependencies

Step 5: Build the Docker Image

Run:

```
docker build -t ml-model-api .
```

Check the image:

```
docker images
```

Step 6: Push the Image to Amazon ECR

First create a repository in Amazon Elastic Container Registry (ECR).
Then authenticate Docker with ECR:

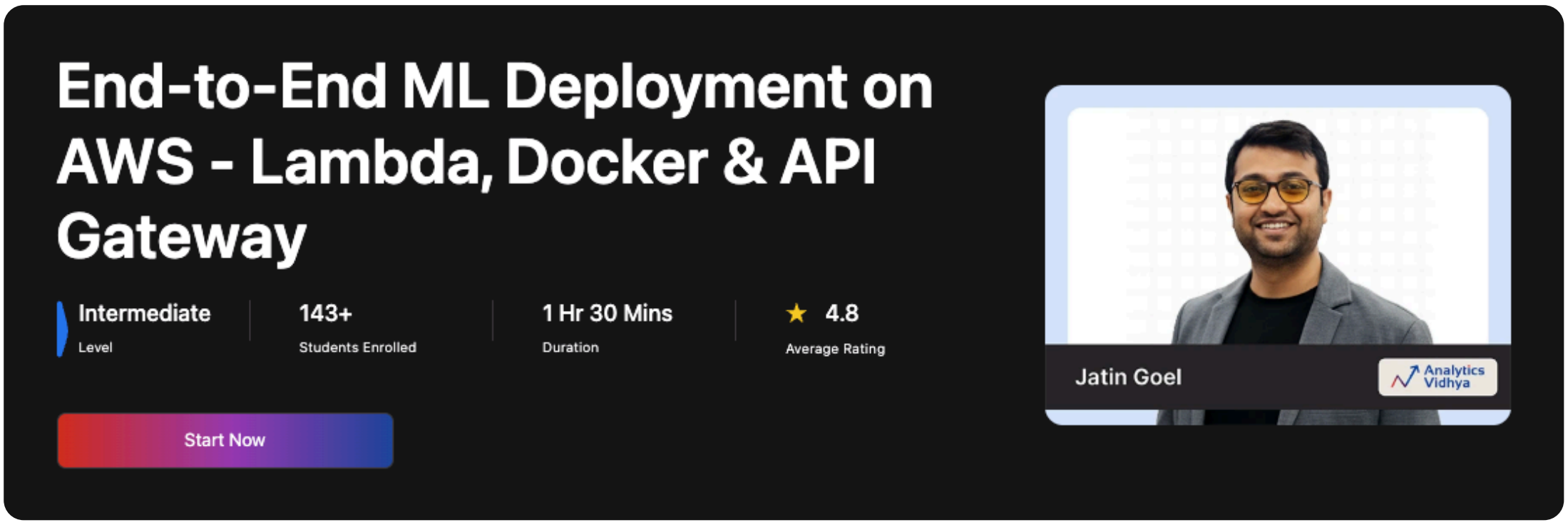
```
aws ecr get-login-password \
--region us-east-1 \
| docker login \
--username AWS \
--password-stdin <account_id>.dkr.ecr.us-east-1.amazonaws.com
```

Tag the image:

```
docker tag ml-model-api:latest \
<account_id>.dkr.ecr.us-east-1.amazonaws.com/ml-model-api:latest
```

Push it:

```
docker push <account_id>.dkr.ecr.us-east-1.amazonaws.com/ml-model-api:latest
```

Your container image is now stored in ECR.

For more information, you can **enroll in this free course**